# Diagnosing Memory-Write Decisions in Conversational Assistants Through Specification-Based Replay

SANGJUN LEE*, Ajou University, Republic of Korea
JIWON YEM*, Ajou University, Republic of Korea
MINJEONG KIM, Ajou University, Republic of Korea
TAEHEON KIM, Ajou University, Republic of Korea
YOUNGCHO SHIN, Ajou University, Republic of Korea
HYOJI HA†, Ajou University, Republic of Korea

Memory-write evaluation must distinguish completing required changes from introducing incorrect writes, yet an end-to-end completion score cannot show which stage held a write. We introduce specification-based replay, which holds saved extractions and proposals fixed, varies the admission rule, and scores the executed whole state, together with exposure: the outputs on which rules can decide differently, counted before scoring, which bound any measurable rule difference. On 48 authored Korean cases, 21 model configurations, and three repetitions (3,024 episodes), replay explains why selected component gains did not carry over to the internally prespecified complete-pipeline comparison with general review, and it yields three findings. First, evidence-checking rules recover 11 corrections on Gemini 2.5 Flash, but releasing every eligible held correction recovers the same 11; the holds originate in empty extracted fact lists, and revising extraction lets the unchanged gate complete five of five selected corrections, against one of five under the original instruction. Second, 12 of 21 configurations have zero exposure to decisions distinguishing the correction rules; only 49 controlled-path outputs from 12 cases reach that boundary. Third, accepting candidate variants of required facts, identified mechanically for subsequent human validation, reduces the gate's avoided reference violations from 82 to 30 and increases lost completed changes from 40 to 92 on saved general-review proposals. On real DSTC2 recognition hypotheses, replay instead locates holds in disagreement or incomplete support across hypotheses. Each mechanism yields a reporting check (trace every hold to its stage, report exposure, and rescore under alternative references), and these checks apply to any memory writer that separates proposal from admission.

CCS Concepts: • **General and reference** → **Evaluation**; • **Computing methodologies** → *Nonmonotonic, default reasoning and belief revision*; *Intelligent agents*; Discourse, dialogue and pragmatics.

Additional Key Words and Phrases: conversational assistants, persistent memory, memory-write decisions, admission control, specification-based evaluation, replay-based diagnosis

## 1 Introduction

Conversational assistants increasingly maintain persistent memories of their users [13]. Updating these memories requires completing requested changes while preserving facts that should remain unchanged. Systems propose additions, revisions, and deletions [4, 8, 29, 48, 49], but downstream answers or final-state scores [26, 46] do not identify which stage produced an error. A correction may be proposed correctly yet withheld because an earlier extraction omitted its supporting fact.

We study this problem by separating generation, admission, and execution. Generation produces extracted facts and proposed operations; admission determines which operations may execute; execution changes the stored state. Our specification-based replay holds the generated artifacts

Sangjun Lee and Jiwon Yem contributed equally to this research.

Authors' Contact Information: Sangjun Lee, Department of Electrical and Computer Engineering, Ajou University, Suwon, Republic of Korea; e-mail: leesj2054@ajou.ac.kr; Jiwon Yem, Department of Business Administration, Ajou University, Suwon, Republic of Korea; e-mail: jieueo310@ajou.ac.kr; Minjeong Kim, Department of Digital Media, Ajou University, Suwon, Republic of Korea; e-mail: ninn@ajou.ac.kr; Taeheon Kim, Department of Business Administration, Ajou University, Suwon, Republic of Korea; e-mail: rlaxogjs0410@ajou.ac.kr; Youngcho Shin, Department of Business Administration, Ajou University, Suwon, Republic of Korea; e-mail: yeongchosin@gmail.com; Hyoji Ha (corresponding author), Humanities Research Institute, Ajou University, Suwon, Republic of Korea; e-mail: hjha0508@ajou.ac.kr.

fixed, applies alternative admission rules, and checks the whole resulting state against an explicit reference. The four rules range from keeping corrections without shared extracted support to releasing eligible held corrections or requiring literal or target-linked evidence. Their *exposure* is the set of outputs on which they can decide differently, determined from gate logs before scoring.

This separation explains a result that an end-to-end score leaves open. In our internally prespecified comparison, fixed in the original protocol but not publicly preregistered, the controlled path did not establish a completion advantage over Review (Section 6.5), although selected admission rules recovered corrections on fixed proposals. The replay analyses, developed after these outcomes had been inspected, show why such component gains need not translate into overall improvement.

The study uses 48 authored Korean cases, 21 model configurations, and three repetitions per case, yielding 3,024 episodes on 48 distinct cases. Its three main findings are (Figure 1):

- *Extraction omissions create recoverable holds.* On Gemini 2.5 Flash, all three rules that relax the agreement gate recover the same 11 corrections, which attributes the recovery to releasing the hold rather than to evidence-based selection. A gate that reads model-extracted evidence inherits the extractor's omissions: in a follow-up on five selected cases, revising extraction lets the unchanged gate complete five corrections, against one under the original instruction (Section 6.1).
- *Zero exposure explains many ties.* Twelve of 21 configurations have zero exposure on controlled-path proposals. Across the panel, only 49 outputs from 12 cases reach the decision boundary. Exposure therefore determines which ties test competing admission decisions (Section 6.2).
- *Reference conventions change the apparent benefit of gating.* Mechanically accepting candidate variants of required facts reduces avoided reference violations from 82 to 30 and increases lost completed changes from 40 to 92 on Review proposals, whereas completed pairwise comparisons among the correction rules are unchanged under both references (Section 6.3).

These findings concern attribution, opportunity, and specification, respectively. Exposure is determined from gate decisions before reference-based scoring, and Proposition 4.1 shows that, restricted to each endpoint population, it bounds the rule differences that any comparison on the saved outputs can reveal. Table 1 places this evaluation alongside eleven memory-write studies. Further analyses show that target-linked evidence blocks person and polarity errors that literal evidence admits, decompose the complete-pipeline comparison into opposite proposal and gate terms, and apply replay to a learned verifier and to real recognition hypotheses (Sections 6.4–6.7).

This article makes three contributions:

- *Specification-based replay with an exposure bound.* A protocol that applies admission rules to fixed generated artifacts, scores executed whole states against an explicit reference, and counts, before scoring, the outputs on which the rules can differ (Sections 4 and 5; Proposition 4.1).
- *Three mechanisms that a completion score hides.* Extraction-induced holds, zero-exposure ties, and reference-dependent gating trade-offs, each traced in saved artifacts across 21 model configurations and contrasted with real speech-recognition hypotheses (Section 6).
- *Reporting checks and a released archive.* One diagnostic check per mechanism (Table 10), together with the cases, references, prompts, raw outputs, executed states, and evaluator needed to apply them (Section 7 and Data, Code, and AI Use).

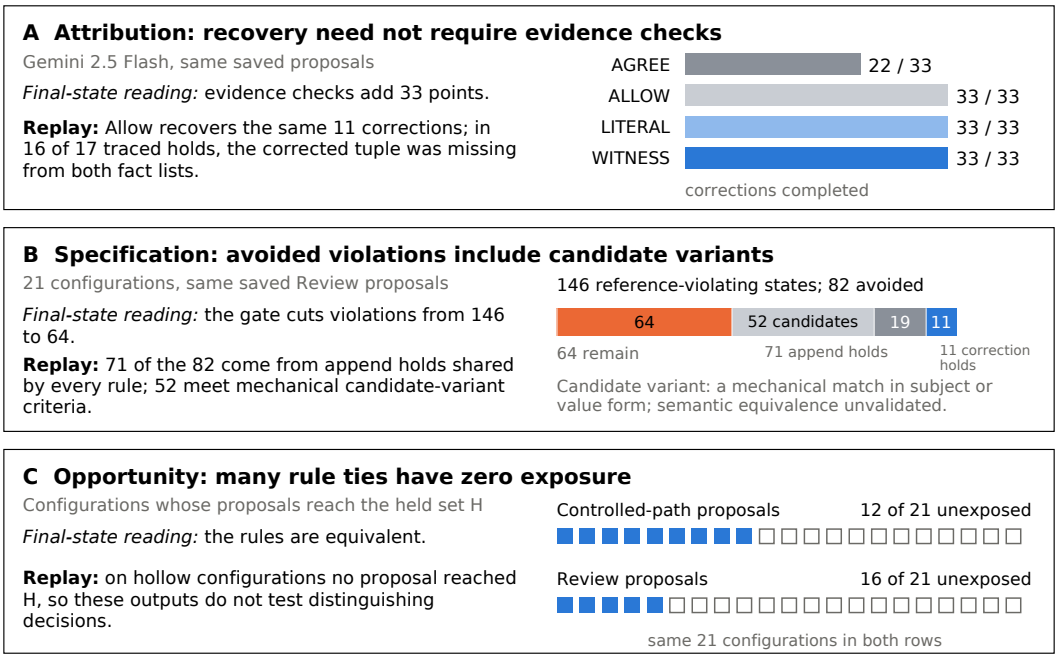



Fig. 1. Three evaluation confounds that a final-state score cannot distinguish: attribution (A), specification (B), and opportunity (C). Each row sets what the final-state score shows beside what replay of the same saved artifacts shows. A: Gemini 2.5 Flash, strict corrections completed out of 33 observations under each rule. B: The 146 reference-violating states under Review on the 21-configuration panel, split by whether the gate avoids them and, if so, by the held operation; "candidate variants" are mechanically identified matches differing in subject label, spacing, or value form (Section 6.3). C: The 21 panel configurations, with controlled-path and with Review proposals; hollow squares mark configurations on which no proposal reached the held set $H$, the corrections that Agree holds solely because the extracted fact lists share no support for them, although the extraction covers every reading (Section 4.1). Outside $H$ all four rules make the same decision.

## 2 Related Work

### 2.1 Memory Updates and Admission in LLM Agents

Several systems give LLM agents a persistent memory that is updated through explicit operations. MemGPT treats the context window as fast memory and moves information to and from external storage through function calls [29]. Mem0 extracts candidate facts and applies one of four operations to existing memories: add, update, delete, or no operation [8]. A-Mem stores memories as linked notes whose attributes are revised as new notes arrive [48], and MemoryBank, recursive summarization, SeCom, LD-Agent, and Generative Agents maintain or select conversational history in related ways [20, 30, 31, 44, 58]. Closest to the correction decision studied here, Bae et al. cast memory management in Korean long-term conversation as pairwise operations between a stored sentence and a new session summary (pass, replace, append, or delete) [4]. They compared trained classifiers with natural language inference models mapped to these operations and evaluated operation accuracy, set-level overlap, and human ratings of later sessions. Memory-R1 learns add, update, delete, and no-operation decisions from a reward based on downstream answer correctness

[49], and Zep invalidates, rather than deletes, knowledge-graph edges that newer information contradicts [34].

A recent line of work treats the write itself as an admission decision. A-MAC learns an admission policy from interpretable factors (future utility, factual confidence, semantic novelty, temporal recency, and a content-type prior) and evaluates admission decisions against labeled candidate memories [54]; ConsistencyGate samples the same model several times to score whether a candidate fact is supported by its source and admits the fact above a threshold [56]; and DerivAudit audits whether a stored memory follows from the history available when it was written, finding that support often lies beyond the writer's citations and that verifiers still admit many unsupported memories [21]. STALE evaluates whether agents notice that a stored memory has been implicitly invalidated by later information [6]. Memory writes are also an attack surface: AgentPoison and MINJA show that poisoned or injected records, the latter inserted through ordinary queries, later steer agent behavior [7, 9]. In our setting, the surrounding history is supplied rather than retrieved, so retrieval failure is not confounded with admission failure; how the position of information affects its use is a separate question [22].

### 2.2 State Revision and Competing Hypotheses

Dialogue-state tracking already models state revision. SOM-DST predicts an operation for each slot before generating values [19], and its oracle analysis attributes most errors to the operation predictor rather than to value generation, an early stage-level diagnosis of the kind we apply to memory writes. AG-DST amends an initially generated state [42], dataflow synthesis supports executable dialogue programs and non-destructive revision [3], and PoA generates semantic-action code that updates the dialogue state [40]. Other trackers select the relevant turns, suppress incorrect values, or rewrite queries to resolve coreference [14, 23, 43]. The second Dialog State Tracking Challenge supplied competing speech-recognition hypotheses for every turn and scored the tracked state turn by turn [16]; its setting is close to ours, although there the tracker produces the state itself, and Section 6.7 replays our admission rules on its hypotheses. MultiWOZ 2.2 removes values that the system offered before the user accepted them [53], and MultiWOZ 2.4 removes values that only the system mentioned [51]; both revisions show why the timing and source of evidence matter. MultiWOZ 2.2 also lists the surface variants of a value as equally correct [53], motivating a sensitivity analysis of matching conventions (Section 6.3). Our comparison intervenes after extraction and proposal are fixed: it evaluates the permission to execute a proposal and thus complements these state-generation methods.

### 2.3 Evidence, Clarification, and Evaluation

Clarification has long been modeled as a decision under uncertainty [10, 12, 32, 52, 55], although human clarification is only weakly tied to annotated ambiguity [25] and is not mirrored by model uncertainty [41]; an uncertain detail can be unnecessary for the next action yet consequential for a durable write. Disagreeing readings resemble the inter-context conflicts studied between retrieved sources [47], and a quoted span shows that a value occurs in the source, not that it supports the proposed fact [1]; this distinction motivates comparing literal evidence with target-linked witnesses. Attribution to identified sources can be defined and rated by people [33], entailment-based checkers detect factual inconsistency [18], LLM judges can approximate human preferences [57], and agreement across readings resembles self-consistency over sampled outputs [45]. $\tau$-bench scores agents by the final database state after tool use [50], the kind of executed-state endpoint that we apply to a single turn. Self-Refine and Reflexion revise outputs at inference time [24, 38]; we use a two-call general review as the alternative complete pipeline, and because the choice of instruction can itself change results [2], every condition receives the same task facts and retention

Table 1. Reporting audit of eleven memory-write evaluations, from what each paper reports. Included are the works of Section 2.1 that make or evaluate per-record decisions to add, revise, delete, forget, or admit memories and report a quantitative evaluation; systems that rewrite one summary or keep all history, and attack studies, are excluded. *Fixed*: the compared write or admission policies act on the same generated candidates. *Both errors*: missed required writes and wrongly admitted writes are reported, not only downstream answers. *Opportunity*: the number of cases on which the compared policies could, or did, differ; this article defines the quantity (exposure, Section 4.2), so a dash records only that it is not reported. *Matching*: how outputs are compared with references; n.r., not reported.

| Work | Write decision scored | Fixed | Both errors | Opportunity | Matching |
|---|---|---|---|---|---|
| MemoryBank [58] | Downstream answers | – | – | – | Human ratings; agreement n.r. |
| MemGPT [29] | Downstream answers | – | – | – | LLM judge and ROUGE-L; agreement n.r. |
| Mem0 [8] | Downstream answers | – | – | – | Token overlap and LLM judge; agreement n.r. |
| A-Mem [48] | Downstream answers | – | – | – | Token and embedding overlap |
| Zep [34] | Downstream answers | – | – | – | LLM judge |
| Memory-R1 [49] | Downstream answers | – | – | – | Token overlap and LLM judge |
| STALE [6] | Downstream probes | – | Partial | – | LLM judge with human check |
| Bae et al. [4] | Operation per memory pair; set overlap | Yes | Partial | – | Human labels, majority of three; 87.5% annotator–gold agreement |
| A-MAC [54] | Admission per candidate | Yes | Yes | – | Candidate labels; process n.r. |
| ConsistencyGate [56] | Admission per fact | Partial | Yes | – | Constructed labels |
| DerivAudit [21] | Support and admission per memory | Yes | Yes | – | Two LLM annotators, blinded LLM adjudication; 96.0% human-review agreement |
| This work | Admission, whole state | Yes | Yes | Yes | Authored references, checked independently by two authors; exact and variant-tolerant matching |

constraints. Automatic and human assessments can agree [39] or rank systems differently [36], so agreement has to be established for each endpoint. Table 1 audits what memory-write evaluations report.

Within the eleven studies in this reporting audit, A-MAC and DerivAudit combine fixed candidates with both error types; exposure, as defined here, is new within this set of reports. Our evaluation adds exposure and reference sensitivity to whole-state comparisons of edits to identified records under competing readings. Because A-MAC, ConsistencyGate, and DerivAudit evaluate candidate labels rather than executed states, Section 6.6 brings the learned-verifier approach into our setting by applying a verifier to the same saved proposals.

Re-evaluations of inference benchmarks, recommenders, and metric learning show how evaluation choices can change conclusions without a new model [11, 15, 28]. Counterfactual replay [37] and performance bounds under incomplete labels [27] provide related methodological context.

Here, exposure complements power analysis [5] by bounding the endpoint difference possible on the saved outputs.

## 3 Task and Decision Boundary

A test case is $x = (h, T, M, \pi)$, consisting of the preceding context $h$; an indexed sequence $T = (t_1, \ldots, t_k)$ of alternative readings of one current utterance, which may coincide; an initial memory state $M$; and a case-level retention permission $\pi$. The readings are simultaneous alternatives for the same speaker's turn, not separate speakers or successive turns. A record has a persistent identifier and a fact tuple (subject, relation, value, time). The model proposes an action $a$, a reply $u$, and an ordered list of memory operations $Q$; the application executes the admitted operations in one transaction and returns the state $M'$ together with an execution log $\rho$.

Four operations are available: APPEND, CORRECT, HOLD, and no-write. A correction targets an existing record and replaces its value while keeping its identifier, whereas a hold leaves a proposed fact uncommitted. Retention permission is separate from factual support, because a clearly stated fact may still be disallowed for storage. Supplying the initial state lets the evaluation distinguish a new fact from an obsolete or protected one without having to infer it. For each case, the reference specifies the allowed actions, the required and forbidden tuples, the records that must remain unchanged, the obsolete versions to retire, and any records that must no longer appear; all of these requirements are checked on the executed state.

We assess required change and preservation together. On a positive case, at least one required fact is absent or an obsolete version must be retired, so holding every proposal fails. A correct target update can still fail whole-state conformity when it also appends a fact outside the requested edit. We score the action and persistent state against the reference and inspect reply–state discrepancies in selected traces; reply truthfulness is not a scored endpoint. In tables, "Viol." abbreviates reference violations. Table 2 defines the terms and endpoints used throughout. A *case* is a distinct input specification and reference; an *episode* is one configuration's run on one case in one repetition. Replaying one saved proposal under four rules creates four evaluated conditions, not four independent episodes.

## 4 Correction Admission Rules

### 4.1 One Proposal and Four Admission Decisions

The controlled path makes two model calls. The first returns an extraction $F$: for each reading, the atomic facts that the reading supports, and, for each pair of a stored record and a reading, one correction witness with a polarity label (affirmed, negated, uncertain, or absent), a new value, and a verbatim quotation. The second call receives the test case together with $F$, marked as fallible, and generates the final proposal. Every admission rule receives the identical test case, extraction, and proposal, together with an isolated copy of $M$, so changing the rule requires no further model call (Figure 2).

Agree applies to every proposed APPEND and CORRECT operation. It admits an operation when the exact subject–relation–value–time tuple is supported by the fact list of every reading or is already present in $M$; otherwise, it changes the operation to HOLD. Agree was the conservative gate of the original protocol, fixed before collection, and is meant to keep a fact out of memory when the readings disagree about it. Shared support requires that the extraction validly cover all readings. Because the test compares tuples exactly as extracted, and because the extraction schema allows a reading's fact list to be empty, a correction can appear in the proposal and in the witnesses yet be missing from the intersection of the extracted fact lists. This is how the held set arises.

Table 2. Terms and endpoints. With three repetitions, the correction, positive, and all-case denominators are 33, 84, and 144 per configuration; with one repetition, they are 11, 28, and 48.

| Term | Definition |
|---|---|
| Reading | One written alternative of the current utterance, supplied as text. |
| Extraction $F$ | Reading-level fact lists and correction witnesses produced by the first model call. |
| Proposal $(a, u, Q)$ | Action, reply, and memory operations produced by the second model call. |
| Held set $H$ | CORRECT operations that AGREE holds only for lack of shared support, although the extraction validly covers every reading. |
| Exposure | Outputs on which the rules can decide differently, that is, outputs that contain an operation in $H$, reported with the number of distinct cases they represent. |
| Complete pipeline | A system that generates its own proposal. REVIEW: a draft plus one general-review call. |
| Controlled path | Extraction, proposal, and WITNESS admission; reply rendered from the execution log. |
| Strict correction completion | Allowed action and exactly the reference-specified whole state on the 11 correction cases. |
| Positive-change completion | Allowed action and exact whole state on the 28 positive cases (the internally prespecified primary endpoint). |
| Reference-violating state | Forbidden or extra write, or damage to a protected fact, as defined by the reference (all 48 cases). |
| Whole-state conformity | Executed state meeting every state requirement of the reference (all 48 cases). |
| Negative preservation | Preservation of the state on the 20 cases that require no change. |

Table 3. Admission rules applied to the same proposal. Outside $H$, every rule keeps AGREE's decision; retention permission and execution checks are shared.

| Rule | Admission within $H$ |
|---|---|
| AGREE | Keeps the conservative hold when shared extracted support is missing. |
| ALLOW | Admits every otherwise eligible correction in $H$. |
| LITERAL | Checks the existing target and the unchanged subject, relation, and time; requires the new value in every reading and current-turn evidence. |
| WITNESS | Additionally requires one affirmed, matching, target-linked witness per reading, supported by an exact source quotation that contains the value. |

Let $H(x, F, Q)$ be the set of operation indices $j$ for which AGREE holds a proposed CORRECT only because its support is not shared, although every reading is validly covered. All differences between AGREE and the other three rules lie within $H$; outside $H$, every rule keeps AGREE's decision (Table 3). ALLOW restores every correction in $H$ and nothing else, so an APPEND that lacks shared support stays held under every rule. Two predicates define the evidence checks. For LITERAL ($L_j = 1$), the proposal must name one existing target; preserve its subject, relation, and time; change its value; and cite the current turn in both the fact and the operation. In addition, the replacement value must occur literally in every reading. For WITNESS ($W_j = 1$), the same target and source conditions must hold, and each reading must supply exactly one witness for the target record that is labeled affirmed, repeats the proposed subject, relation, time, and value, cites the current turn, and quotes a verbatim span of that reading containing the replacement value. The application verifies the

structure and linkage of these model-generated witnesses, but the polarity label itself is taken from the model.

Two examples show how the checks differ. The sentence "the shop opened in 1993, not 1991" satisfies both checks for an opening-year record. By contrast, "My son works at Daehan Post" satisfies the literal check for an update of the speaker's own workplace but should fail the witness check, which requires every reading to affirm the value for the targeted record: the affirmed statement concerns the son.

### 4.2 Containment and the Opportunity to Intervene

The admitted sets of the four rules are nested. Write $A_X$ for the indices in $H$ that rule $X$ admits; then

$$A_{\textsc{Witness}} = \{\, j \in H : W_j = 1 \,\} \;\subseteq\; A_{\textsc{Literal}} = \{\, j \in H : L_j = 1 \,\} \;\subseteq\; A_{\textsc{Allow}} = H. \tag{1}$$

The first inclusion holds because a passing witness carries the required target and source linkage and an exact quotation that contains the value, so the value occurs in every reading; the second follows from the restriction to $H$. This ordering concerns admitted operations only. It need not carry over to final-state accuracy when a transaction is rejected, when operations interact, or when a witness is wrong.

Agree and Allow are therefore the two ends of this ordering rather than competing designs: no rule that acts on $H$ can admit more held corrections than Allow or fewer than Agree. We compare Literal and Witness with both ends: with Allow for the reference violations they prevent, and with Agree for the corrections they recover and the reference violations they add. Because Agree reads model-extracted facts, its holds can originate in extraction rather than in disagreement between readings; the saved extraction shows which of the two occurred (Section 6.1). Witness and Literal decide differently only on $K = \{\, j \in H : L_j = 1,\ W_j = 0 \,\}$, where a hold may either protect the correct state or keep an obsolete fact unnecessarily. When $H$ is empty, all four rules must agree.

Admission and execution are deterministic given the saved case, extraction, and proposal. Replay therefore identifies effects conditional on these artifacts, rather than effects of regenerating a complete pipeline [17]. For the four correction rules, outputs with nonempty $H$ contain every opportunity for different admission decisions. A particular rule pair may differ on only a subset; different decisions can also yield the same state or endpoint score. Exposure is counted from saved artifacts before scoring.

Proposition 4.1 (Exposure bound and reference dependence). *Let rules $X$ and $Y$ act on the same endpoint population $D$, with $N_D = |D|$, and let $E_D \subseteq D$ contain every output on which the rules can make different admission decisions. For every endpoint indicator $S$ computed on the executed state, (i) $|\bar{S}_X - \bar{S}_Y| \le |E_D|/N_D$; (ii) the difference $\bar{S}_X - \bar{S}_Y$ depends on the reference only through the scores of the outputs in $E_D$; and (iii) if a second reference changes $S$ on at most $C$ of the $2|E_D|$ output–rule pairs in $E_D$, the difference changes by at most $C/N_D$.*

Proof. Outside $E_D$, the two rules admit the same operations, so the deterministic executor produces the same state and $S$ takes the same value under $X$ and $Y$. Hence $N_D(\bar{S}_X - \bar{S}_Y) = \sum_{o \in E_D}(S_X(o) - S_Y(o))$, whose absolute value is at most $|E_D|$. The sum involves the reference only through outputs in $E_D$, and a change to at most $C$ individual scores changes the sum by at most $C$. □

The exposure set and denominator must refer to the same endpoint population. Thus, 49 exposed outputs among 3,024 episodes bound an all-case endpoint difference by approximately 1.6 percentage points, not automatically a positive-change or correction-completion difference. On Gemini 2.5

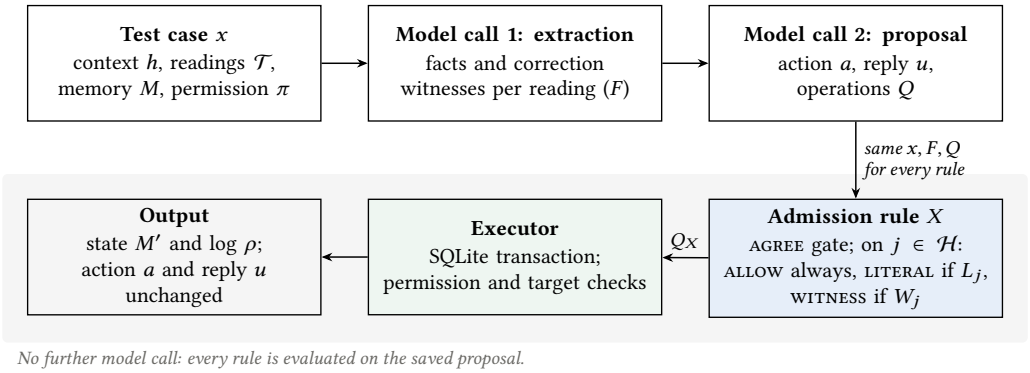



Fig. 2. Fixed-proposal architecture. Two generation calls produce one extraction and one proposal. Each admission rule is then applied to the same artifacts and executed on an isolated copy of the initial state.

Flash, 11 exposed outputs among 144 permit a 7.6-point all-case difference, whereas 11 among 33 correction observations permit a 33.3-point correction difference. For gating versus ungated execution, exposure must also cover affected appends and decisions outside $H$. Reference review must use the relevant gate-log-based set, including exposed outputs that tie under current scores (Section 6.3).

### 4.3 Execution and Complete Pipelines

The application applies operations transactionally to a separate SQLite state for each rule and records read-after-write execution logs; the executor enforces retention permission and target existence for every rule. Because admission does not regenerate the action or the reply, differences in state outcomes between rules can be assigned to admission alone within this pipeline. For the same reason, a held write can coexist with a reply that asserts the held value (Appendix H), and the unchanged reply is not credited to admission. WITNESS uses the same two calls per output as AGREE, and ALLOW and LITERAL are computed from the saved proposals (Algorithm 1; Appendix C).

The REVIEW pipeline generates its own proposal from a draft and one general-review call, so it and the controlled path use the same number of calls. The original protocol also ran a single direct draft and one draft–feedback–refine cycle adapted from Self-Refine [24]; they share REVIEW's initial draft where the design permits and are reported in Appendices A and F. The internally prespecified primary test compares the controlled path, with its reply rendered from the execution log, with REVIEW. Rendering does not change any state endpoint, so the tables report the common state score under WITNESS (Appendix A).

## 5 Evaluation Design

### 5.1 Sources and Model Configurations

The main source contains 60 Korean test cases that the authors designed and verified, with case text drafted by an AI tool to the authors' specifications: 12 development cases and 48 held-out cases spanning 12 problem families of four cases each (Appendix A, Table A3). Each held-out case was run three times per model configuration. A private reference, fixed before the main collection, specifies the requirements listed in Section 3, including the distinction between replacing a value and removing its record. The two co-first authors independently checked each of the 48 held-out references against its case; external annotation of these references is the first item of future work

**Algorithm 1** Admission on a fixed proposal. Reply rendering is a separate downstream option.

**Require:** case $x = (h, T, M, \pi)$, extraction $F$, proposal $(a, u, Q)$, rule $X$ (AGREE, ALLOW, LITERAL, or WITNESS)
**Ensure:** action $a$ and reply $u$ unchanged; executed state $M'$ and log $\rho$

**if** $Q$ fails schema validation **then**
  **return** $(a, u, M, \emptyset)$ ▷ no write; scored as an invalid output
**end if**
$Q_X \leftarrow \text{AGREEGATE}(x, F, Q)$ ▷ holds APPEND and CORRECT without shared support
**for all** $j \in H(x, F, Q)$ **do**
  **if** $X = \text{ALLOW}$ **or** $(X = \text{LITERAL}$ **and** $L_j = 1)$ **or** $(X = \text{WITNESS}$ **and** $W_j = 1)$ **then**
    $Q_{X,j} \leftarrow Q_j$ ▷ restore the proposed correction
  **end if**
**end for**
$(M', \rho) \leftarrow \text{EXECUTE}(M, Q_X, \pi)$
**return** $(a, u, M', \rho)$

(Section 9). Held-out cases were kept separate from the 12 development cases. The collection script received the public cases but not the reference, and the original configurations, prompts, admission rules, and evaluator were fixed before the original collection. We use Korean because widely used memory benchmarks are in English [26, 46] and because Korean writing yields several surface forms of one fact (word spacing varies in practice, and a person may be named by a kinship term, a given name, or both), which stresses exact tuple matching (Section 6.3).

All alternative readings of the main source are written text, drafted to the same author specifications. Within a case, they can be identical, differ in surface form, or differ in value, person, or polarity; they play the role of competing recognition hypotheses, so how often they reach the held set characterizes this written source; Section 6.7 supplies hypotheses from a real recognizer.

The original protocol planned a Holm-adjusted primary comparison for five model families; Gemini 2.5 Flash, Llama 3.1 8B, and Qwen3 14B (local 4-bit) were collected, and two families were not (Appendix C). Six later Gemini configurations reuse the 48 cases and were added after the earlier results had been inspected. The model panel comprises the 21 configurations that completed all 48 cases in all three repetitions with at least 95% valid final outputs in both REVIEW and the controlled path: seven Gemini, eight Claude, and three GPT configurations together with DeepSeek 3.2, GLM 5, and Qwen3 14B, for 3,024 case-repetition episodes. This validity criterion was set after collection and applies to complete pipelines, not semantic outcomes; of the 28 configurations with complete triplets, seven fall below it, among them Llama 3.1 8B (Appendix G). Model labels denote recorded provider or runtime configurations and do not imply matched parameter counts, precision, or reasoning budgets.

Table 4 lists the further sources. Three authored or adapted sources probe transfer and are reported in Appendices D–F: a 24-case contrast set that the authors specified after the initial finding and that a separate AI agent drafted without access to the rule code or earlier outputs, a 12-case boundary probe, and 16 selected MultiWOZ 2.4 turns [51]. A fourth source, 48 user turns from the DSTC2 test set, supplies real speech-recognition hypotheses with a reference derived from human-reviewed labels (Section 6.7). Three diagnostic follow-up procedures, designed after the earlier results had been inspected and fixed before their own outputs, test the diagnoses (Appendix J): a revised extraction instruction on the held cases, a learned verifier (Gemini 3.1 Pro Preview) that approves or holds each APPEND and CORRECT in saved REVIEW proposals without seeing the extraction or the reference, and the DSTC2 replay.

Table 4. Sources and their roles. Additional configurations reuse the main cases, and each source keeps its own denominators.

| Source | Cases | Repetitions | Role |
|---|---|---|---|
| Main Korean cases | 48 | 3 | Fixed-proposal and complete-pipeline comparisons |
| DSTC2 test turns | 48 | 1 | Real recognition hypotheses; 41 callers |
| Contrast set | 24 | 1 | 12 supported and 12 unsupported edits |
| Boundary probe | 12 | 1 | Six paired edits; no exposure |
| Public dialogue probe | 16 | 1 | 10 eligible self-repairs; six unscored |
| Follow-up runs on main cases | 5; 48 | 1 | Revised extraction; learned verifier |

## 5.2 Endpoints and the Variant-Tolerant Reference

The primary endpoint internally prespecified in the original study is positive-change completion: joint success of the action and the whole state on the 28 of the 48 main cases that require a new fact or the retirement of an obsolete version. A successful output must be valid, choose an allowed action, contain the required facts, omit forbidden and obsolete tuples, preserve protected facts, satisfy the required removals, and introduce no unlisted fact; provider failures and invalid outputs stay in the denominator. Strict correction completion applies the same criterion to the 11 correction-eligible cases. The reference-violation endpoint flags forbidden or extra writes and damage to protected facts across all 48 cases. Successful positive changes and reference-violating states are always reported together, so that withholding more operations is not mistaken for improving the task.

The reference matches tuples exactly, so an unlisted fact counts as a violation even when it restates a required fact in another form. To measure how much each conclusion depends on this convention, we re-scored every saved output under a *variant-tolerant reference*. A tuple counts as a required or protected fact when its relation and time match and, after Unicode NFC, case folding, and whitespace removal, its subjects are equal, both denote the participant, or one contains the other, and its values are equal or one contains the other; a participant label is never matched to another person's label. Forbidden and obsolete tuples stay exact, so the second reference can only credit outputs. It accepts 38 distinct pairs in 10 cases, all listed in Appendix K, and the scorer reproduces every archived score of the exact reference. These are candidate equivalences for human validation, because substring containment can also match different entities or meanings. The exploratory re-scoring measures how much each conclusion depends on the matching convention; neither reference is presumed more accurate, and human judgments may fall outside both.

## 5.3 Paired Analysis

For configuration $m$ and case $i$, the paired difference is $d_{mi} = \frac{1}{3}\sum_{r=1}^{3}\left[Y_{mir}(\text{ctrl}) - Y_{mir}(\textsc{Review})\right]$, where $Y$ is the endpoint indicator for repetition $r$ and ctrl denotes the controlled path; the effect estimate averages $d_{mi}$ over cases. Uncertainty is reported as 95% percentile intervals from 5,000 case-cluster bootstrap draws that keep the repetitions, and for pooled estimates the configurations, of a case together (48 clusters for all-case endpoints, 28 for positive completion, and 11 for correction completion). The internally prespecified exact two-sided McNemar test counts a case as a success when it succeeds in at least two of three repetitions, and the original plan adjusted the five planned family tests with Holm's method (Appendix A). Each original-protocol run made $48 \times 3 \times 6 = 864$ physical calls, of which each two-call pipeline used 288, without retries. Gemini 2.5 Flash used temperature 0.2, disabled thinking, and 2,048 output tokens; later configurations used their supported thinking settings, so the panel compares configurations as run rather than at equal

Table 5. Status of the reported analyses. "Prespecified" refers to the original internal protocol.

| Analysis | Inferential role |
|---|---|
| Controlled path versus Review (original families) | Internally prespecified primary test; not publicly preregistered; five-slot Holm accounting retained. |
| Agree versus Witness | Planned secondary contrast; descriptive. |
| Allow, Literal, hold traces, exposure; gate on Review proposals | Post hoc, on saved artifacts. |
| Variant audit, variant-tolerant re-scoring, decomposition | Post hoc and mechanical; no model call. |
| Later Gemini configurations; 21-configuration panel | Exploratory reuse of the same 48 cases. |
| Further sources and follow-up runs | Diagnostic; informed by earlier findings and fixed before their own outputs. |

compute (Appendix C). Because configurations and repetitions share cases, distinct cases are reported alongside pooled counts.

### 5.4 Prespecified and Exploratory Analyses

The study began with an internally prespecified comparison of the controlled path and Review; the protocol was fixed internally before the original collection. Replay analyses were developed after the initial results were inspected. They explain those outcomes, while the original test keeps its inferential status (Table 5). Applying the gate to saved general-review proposals pairs each saved Review proposal with the extraction from the same configuration, case, and repetition, applies one of the four rules, and executes the result with the frozen executor; we write Review+Witness and so on. These conditions, two string-normalization controls, the variant audit, and the variant-tolerant re-scoring required no model call (Appendices I and K).

## 6 Results

We first report the three findings from replay on fixed extractions and proposals (Sections 6.1–6.3), then target-linked evidence (Section 6.4), the decomposition of the complete-pipeline comparison (Section 6.5), and two applications of replay beyond the main rules and source (Sections 6.6 and 6.7).

### 6.1 Finding 1: Extraction Omissions Create Recoverable Holds

On Gemini 2.5 Flash, the agreement gate withheld 11 required corrections that the other three rules admitted. Changing admission from Agree to Literal or Witness increases completed corrections from 22 to 33 of 33 observations, while reference-violating states remain at 3 of 144 under every rule (Table 6). The 11 recovered corrections come from five of the 11 eligible cases, and eight are repetitions of self-repair cases. The paired case-mean gain is 33.3 percentage points (95% case-cluster interval 12.1 to 57.6), and the case-level McNemar test, with four discordant cases all favoring Witness, gives $p = .125$; four discordant cases are too few to resolve the gain, so we report it as a count of cases (Section 8). Because Allow recovers the same corrections without the evidence predicates, releasing the hold is sufficient on these proposals, and the recovery is attributable to that release rather than to evidence-based selection.

In all 11 held observations, both extracted fact lists were empty, although the correction appeared as two affirmed witnesses matching the proposal. In the other 22 observations, both fact lists contained the corrected tuple. Thus the holds arose from extraction omissions, not conflicting readings. The contrast set contains five further supported corrections with the same omission pattern, all

Table 6. Comparisons on identical proposals in the seven Gemini configurations, in the order AGREE / ALLOW / LITERAL / WITNESS, under the exact reference. Strict correction uses 33 observations from 11 cases, and the reference-violation endpoint uses 144 observations from 48 cases. The $H$ column reports exposed outputs and, in parentheses, distinct cases; by Proposition 4.1, no two rules can differ by more than this count. All counts combine three repetitions.

| Configuration | Correction /33 | Viol. /144 | $H$ outputs (cases) |
|---|---|---|---|
| Gemini 2.5 Flash | 22 / 33 / 33 / 33 | 3 / 3 / 3 / 3 | 11 (5) |
| Gemini 2.5 Pro | 31 / 31 / 31 / 31 | 3 / 9 / 4 / 3 | 6 (3) |
| Gemini 2.5 Flash-Lite | 31 / 31 / 31 / 31 | 16 / 19 / 16 / 16 | 3 (1) |
| Gemini 3 Flash Preview | 33 / 33 / 33 / 33 | 12 / 12 / 12 / 12 | 0 (0) |
| Gemini 3.1 Pro Preview | 33 / 33 / 33 / 33 | 2 / 2 / 2 / 2 | 0 (0) |
| Gemini 3.5 Flash-Lite | 32 / 32 / 32 / 32 | 7 / 12 / 7 / 7 | 5 (2) |
| Gemini 3.8 Flash | 33 / 33 / 33 / 33 | 1 / 1 / 1 / 1 | 0 (0) |

recovered by the three releasing rules. Only one of the 17 traced holds reflects disagreement: one reading negates the new residence (Appendix D).

A follow-up re-extracted one held repetition from each of the five cases under the original instruction and a revision that also lists affirmed corrections among the extracted facts (Appendix J). The saved proposal and AGREE gate were unchanged. The original instruction completed one of five corrections; the revision completed all five, with no reference violation in either arm. CT01 recovered spontaneously under the original instruction and is not an effect of the revision. On Gemini 3.8 Flash, which had zero exposure, both instructions produced identical outcomes on all 48 cases. The revised extractor was sufficient to remove the selected Flash holds. A gate that reads model-extracted evidence thus inherits the extractor's omissions, so a hold has to be traced to its source before it is credited to admission (Table 10).

### 6.2 Finding 2: Zero Exposure Explains Many Ties

Gemini 3 Flash Preview, 3.1 Pro Preview, and 3.8 Flash produce no output that reaches $H$ (Table 6). Their ties are instances of the opportunity confound, as is the 12-case boundary probe, in which no output reached $H$ and every rule produced the same states (Appendix E). Across the 21 panel configurations, the saved gate decisions on controlled-path proposals show no exposed output on 12 configurations and 49 exposed outputs, from 12 cases, on the other nine (Appendix G). Because every configuration answers the same 48 cases, the range of exposure, from 0 to 11 of 144 outputs, shows that whether this benchmark can separate admission rules depends on the generator; on REVIEW proposals, 16 of the 21 configurations have no exposed output (Appendix I). Exposure therefore has to be reported per configuration, not per benchmark. By Proposition 4.1, no panel-wide comparison among the rules on these proposals can move an all-case endpoint by more than 49 of 3,024 outputs, or 1.6 percentage points. The same count tells benchmark designers what to report: the rules can differ on only 12 of these 48 cases (Section 8). Both evidence checks release 12 of the 49 holds, each completing a required correction (11 on Gemini 2.5 Flash and one on DeepSeek 3.2); LITERAL alone releases two more, both reference violations; and both evidence checks keep the other 35, all in cases that require no change, where the held state conforms to the reference. On most exposed outputs, the agreement gate thus withheld writes that the reference rejects, as designed.

Within the exposed outputs, a literal check supplies most of the protection that the evidence checks add. Relative to ALLOW, WITNESS avoids 14 reference-violating state observations and

Table 7. The gate on saved Review proposals under the exact and the variant-tolerant reference (21 configurations, 48 cases, three repetitions). Strict correction counts (690, 676, 688, 688, and 688 of 693) are the same under both references. The last column counts held decisions among 1,087 proposed APPEND and 762 proposed CORRECT operations.

| | Viol. /3,024 ↓ | | Positive /1,764 ↑ | | |
|---|---|---|---|---|---|
| **Saved path** | **Exact** | **Variant-tol.** | **Exact** | **Variant-tol.** | **Held APPEND / CORRECT** |
| Review | 146 | 44 | 1,623 | 1,725 | – |
| Review + Agree | 64 | 14 | 1,571 | 1,621 | 108 / 23 |
| Review + Allow | 75 | 25 | 1,583 | 1,633 | 108 / 0 |
| Review + Literal | 65 | 15 | 1,583 | 1,633 | 108 / 10 |
| Review + Witness | 64 | 14 | 1,583 | 1,633 | 108 / 11 |

Literal 13, all on Gemini 2.5 Pro, 2.5 Flash-Lite, and 3.5 Flash-Lite and from three distinct cases, while both keep Allow's strict-correction count on every configuration (Table 6). On the saved Qwen3 14B proposals, Allow produces 29 reference-violating states out of 144 and Literal and Witness 20 each, with strict correction at 33 of 33. Within the seven Gemini configurations, Witness matches Agree's reference-violation count while recovering the 11 Flash corrections; the panel exposure summary also includes one recovered correction on DeepSeek 3.2.

### 6.3 Finding 3: Reference Conventions Change the Gating Trade-off

Applying the gate to saved Review proposals more than halves reference-violating states under the exact reference, from 146 to 64 with Witness, at a cost of 40 positive changes (Table 7). The two bounds separate the parts of the gate: Review+Agree already reaches 64, and Review+Allow 75. Counting decisions by operation locates the difference (Appendix I). All four rules treat APPEND identically and hold 108 of 1,087 proposed appends; these holds account for 71 of the 82 avoided reference violations and 36 of the 40 lost positive changes, and the other four losses come from invalid extractions. Agree holds 23 of 762 proposed corrections, on five configurations; Allow releases all of them, Literal 13, and Witness 12, and each release shared by these rules completes a required correction. On these proposals, the gate acts mainly as an append policy.

An audit of the avoided reference violations shows what they consist of. Comparing each held tuple with the states that conform to the reference for the same case, 13 of the 82 involve a forbidden tuple, and 52 of the remaining 69 are mechanically identified candidate variants of a required fact with different spacing (6), subject label (29; e.g., "어머니 순자" ("mother Sunja") for "순자"("Sunja"), or value form (17; e.g., "자전거 타기" ("riding a bicycle") for "자전거"("bicycle")). Under exact matching, these 52 observations lack the required fact with or without the gate, so the hold lowers the violation count without changing completion.

The variant-tolerant reference measures how much this matters (Table 7). Under it, Review produces 44 reference-violating states instead of 146 and 1,725 positive changes instead of 1,623: 102 previously violating states become conforming under the mechanical matcher. The gate then avoids 30 violations instead of 82 (paired difference 0.99 rather than 2.71 percentage points over 3,024 observations; case-cluster interval 0.40 to 1.69), and its cost rises from 40 to 92 positive changes (−5.22 points over 1,764, interval −8.22 to −2.49), because the appends it holds are now credited as required facts; whole-state conformity falls by 79 observations rather than 27. The reference therefore changes the apparent balance between protection and missed changes. The sensitivity is not peculiar to our references: MultiWOZ 2.2 lists the surface variants of a value as equally correct for the same reason [53]. The two conventions represent opposite representation choices: the exact

reference lists one form per fact, while the variant-tolerant reference accepts mechanically defined equivalences, which human validation will adjudicate (Section 9).

For the completed replay comparisons available in the archive, reported pairwise score differences among the correction rules are unchanged under the two references. Nonzero differences occur on 36 controlled-path and 23 review-proposal outputs; exposure is still counted from gate logs (49 and 23 outputs), because tied scores can hide different decisions. Fourteen controlled-path Allow replays are missing, and an additional exposed output has tied endpoint scores (Appendices G and K). Completing these replays will extend the invariance check to the whole panel. For Review versus Review+Witness, the reference changes the difference on 52 of the 127 outputs whose endpoints differ. Normalizing strings inside the gate changes no executed state on the main panel, whereas case differences affect the public task-dialogue probe (7 completed self-repairs under Allow versus 4 under the evidence checks, out of 10 eligible; Appendix F).

### 6.4 Target-Linked Evidence Blocked Person and Polarity Errors

The one reference-violating state that the literal check admits on Gemini 2.5 Pro illustrates what the witness check adds (Figure 3). The utterance describes a son's workplace, and the same proposal both appends the son's workplace and corrects the participant's stored workplace. Because the replacement value occurs in both readings, Literal, like Allow, admits the second update; the witness for the participant's record is not affirmed, so Witness holds it and keeps the participant's workplace. Target-linked witnesses can thus block a value that appears in every reading but belongs to another person. The contrast set supplies a second situation, a value that one reading negates (Section 6.1), and the third is an uncertain report about a travel destination with a request not to change it (CT11), which Literal admits and Witness holds on GLM 5 controlled-path proposals and on saved Qwen3 14B review proposals. Literal occurrence cannot tell whose fact a value is or whether a reading affirms it; binding the evidence to the target record can, and in all four observations, which represent three situations, it changed the executed state. Section 6.7 adds a fifth, a negated value among real recognition hypotheses.

### 6.5 The Complete-Pipeline Difference Is Two Opposite Terms

Under the exact reference, complete-pipeline differences split evenly across configurations. Among the 21 panel configurations, the controlled path increases positive-change completion relative to Review on 9, decreases it on 9, and ties on 3, and reference violations decrease on 12, increase on 7, and tie on 2; 11 configurations change direction across repetitions (Appendix G). The internally prespecified primary comparison on the original families did not establish a completion advantage; this does not demonstrate equivalence between pipelines. On Gemini 2.5 Flash, Review completes 82 of 84 positive-change observations and the controlled path 80 (difference −2.4 percentage points, 95% interval −6.0 to 0.0; McNemar $p = 1.00$); on Qwen3 14B the difference is −8.3 points (−23.8 to 6.0); and Llama 3.1 8B, outside the panel because its controlled path produced 68 invalid final outputs, completes 6 of 84 against 29 (−27.4 points, −44.0 to −10.7; Holm-adjusted $p = .0586$).

An intermediate replay condition decomposes the observed difference: Review+Witness applies the controlled path's extraction and gate to the saved review proposal. The complete-pipeline difference equals the controlled path minus this intermediate condition (proposal term), plus the intermediate condition minus Review (gate term; Table 8). This is a conditional decomposition through the chosen intermediate condition; it does not estimate a full factorial interaction. Under the exact reference, the controlled path's proposals add 67 reference violations and 17 positive changes relative to review proposals under the same gate (violations +2.22 points, interval 1.03 to 3.67), and the gate removes 82 violations and 40 positive changes; the two terms nearly cancel, leaving −15 and −23. Review also completes all 33 strict corrections on Gemini 2.5 Flash, so the

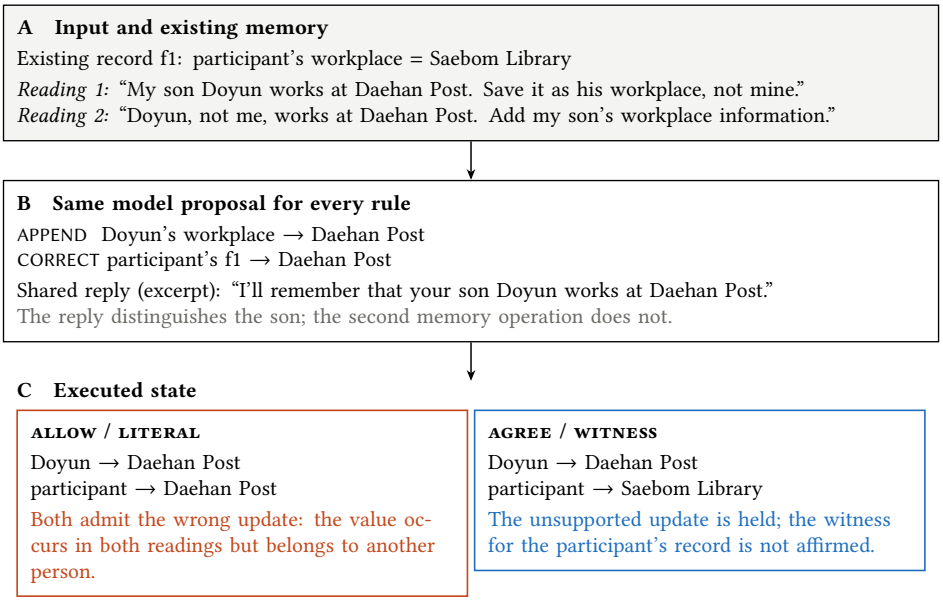



Fig. 3. Target binding changes the executed state. LITERAL accepts a workplace value that appears in both readings, whereas WITNESS rejects assigning the son's workplace to the participant. The reply and the memory operation are verified separately.

Table 8. Exact decomposition of the complete-pipeline difference, controlled path minus REVIEW, pooled over 21 configurations, 48 cases, and three repetitions (counts of observations; reference violations over 3,024, positive changes over 1,764). The proposal term compares two proposals under the same extraction and WITNESS gate; the gate terms attribute each paired change to the held operation, and no observation changed both.

| | Reference violations | | Positive changes | |
|---|---|---|---|---|
| **Term** | **Exact** | **Variant-tol.** | **Exact** | **Variant-tol.** |
| Proposal: controlled path − REVIEW+WITNESS | +67 | +40 | +17 | +44 |
| Gate: held APPEND | −71 | −19 | −36 | −88 |
| Gate: held CORRECT | −11 | −11 | 0 | 0 |
| Gate: invalid extraction | 0 | 0 | −4 | −4 |
| Complete pipeline: controlled path − REVIEW | −15 | +10 | −23 | −48 |

fixed-proposal gain has nothing to recover there. Under the variant-tolerant reference, the proposal term is +40 violations and +44 positive changes and the gate term −30 and −92; the controlled path then completes fewer positive changes on 13 configurations and more on one (pooled −2.72 points, −4.59 to −1.02), and violations decrease on 6 configurations and increase on 6. The direction of the complete-pipeline comparison thus depends on the reference, and the decomposition shows why: the gate's append holds, not the extraction-informed proposals, carry the completion cost.

Table 9. Admission rules on real recognition hypotheses (DSTC2, Gemini 3.1 Pro Preview, one run). Goal conformity requires the exact food, area, and price-range goal; required changes count the 36 turns whose goal changes; a new wrong write introduces a value absent from the official goal.

| Rule | Goal conformity /48 ↑ | Required changes /36 ↑ | New wrong writes /48 ↓ |
|---|---|---|---|
| Ungated proposal | 33 | 23 | 3 |
| Allow | 33 | 23 | 3 |
| Agree | 18 | 6 | 0 |
| Literal | 17 | 6 | 1 |
| Witness | 18 | 6 | 0 |

### 6.6 Replay Locates Where a Learned Verifier Acts

A learned verifier [57] replaced the gate on one saved Review proposal per case for three configurations (144 proposals; Appendix J). On Gemini 2.5 Flash it matches Review+Witness on every endpoint, including five more completed corrections than Review+Agree; these are the extraction-induced holds of Finding 1. Its other differences occur outside $H$: it admits a required append on Gemini 3.8 Flash and blocks two writes on Qwen3 14B, reducing reference violations from three to one. All paired intervals against Review+Witness include zero. Replay thus separates the verifier's decisions into those inside $H$, where on Gemini 2.5 Flash it matches Review+Witness, and those outside $H$, which the correction rules never reach. The 144 calls averaged 4.39 seconds and about 850 tokens each.

### 6.7 On Real Recognition Hypotheses, Replay Finds a Different Cause

Unlike the written readings of the other sources, the DSTC2 source supplies hypotheses that a speech recognizer produced [16]. One user turn was selected from each of 48 test dialogues (41 callers) by a hash-ranked rule fixed before any output: 24 goal revisions, 12 denials without an explicit replacement, and 12 turns with recognition errors, a stratum selected with the labels and reported separately. The readings are the 433 distinct non-empty hypotheses of the live 10-best lists, in rank order and without likelihood weights. The initial memory is the previous official goal for food, area, and price range; the reference is the current official goal, derived from human-reviewed semantic labels and matched exactly; and the current transcript and labels were withheld from the model. The two-call pipeline, with the three slots in place of personal relations and the gate unchanged, ran once on Gemini 3.1 Pro Preview, all 48 outputs valid (Table 9); a partial Gemini 3.8 Flash run is reported in Appendix J.

Exposure is high on this source: Agree held a proposed correction in 20 of the 48 outputs, against at most 11 of 144 for any main-source configuration. The trace also returns a different attribution. None of the 20 holds shows the pattern of Section 6.1, affirmed witnesses in every reading with the tuple missing from the fact lists; in ten, different hypotheses yield different extracted values, and in the other ten the value is supported by only some hypotheses or omitted from some. Releasing every hold (Allow) completes 17 required changes and introduces 3 wrong values. Neither evidence check releases any of the 17, because both require the value in every hypothesis, and Literal releases one hold, a wrong write. Relative to Allow, Witness lowers goal conformity by 31.25 percentage points (95% caller-cluster interval 16.3 to 45.8) and required-change completion by 47.2 points (32.3 to 62.9), and it lowers turns with a new wrong write by 6.25 points (0.0 to 13.5).

Two turns show the trade-off. In DSTC2_45, the caller says "i dont want chinese i want thai food"; the stored goal is already thai, but every hypothesis contains "chinese," and the proposal changes the goal to chinese. Literal admits the change because the value occurs in every hypothesis,

Table 10. Implications of the three findings. The first row has been tested (Section 6.1); the other rows define follow-up evaluations.

| Finding | Diagnostic check | Implication |
|---|---|---|
| Extraction omitted supported corrections | Compare with ALLOW; inspect saved fact lists | Revise extraction; test the fixed revision on new cases |
| Rules tied with zero exposure | Count outputs reaching $H$ before scoring | Add cases that distinguish the rules; report the enriched sampling design |
| Reference conventions changed the gating trade-off | Re-score the same executed states | Validate fact equivalences before judging the append policy |

whereas WITNESS, like AGREE, holds it for lack of affirmed, target-linked witnesses: the polarity error of Section 6.4 on a real recognizer. In DSTC2_07, "how about vietnamese food" appears in some hypotheses as "how about you need food"; ALLOW admits the correct change, and the three strict rules hold it. On this source, requiring support in every hypothesis prevents wrong writes but also withholds recoverable changes. Because the Korean and DSTC2 sources differ in language, domain, reference construction, number and origin of readings, and sampling, the contrast does not isolate recognition noise; what transfers is the diagnostic procedure, which on this source attributes the holds to a different cause.

## 7 Discussion

### 7.1 Interpreting Rule Comparisons

The three findings call for different checks. An ALLOW comparison and the saved extraction can distinguish evidence-based selection from recovery of upstream omissions. Exposure determines whether a tie tests competing decisions. Re-scoring tests whether a reported benefit depends on the reference's representation of facts. Because each check answers a different question, together they locate a reported gain in generation, in admission, or in the reference.

### 7.2 From Diagnosis to Action

Table 10 links the three findings to their implications. The extraction revision has already been tested on five selected cases (Section 6.1); the other two rows define the next evaluations.

The DSTC2 result sets a boundary on the first diagnosis: there, holds involved disagreement or incomplete support across recognizer hypotheses, rather than the extraction pattern in the main source. Majority or likelihood-weighted support is a candidate intervention, with thresholds fixed on development data.

An external-writer replication should freeze artifacts at a native interface. For example, Mem0 separates extraction from ADD, UPDATE, DELETE, and NOOP decisions [8]; a comparison could fix extracted candidates, retrieved memories, record identifiers, and the initial state while varying the update mechanism. This would test the diagnostic procedure on another architecture, but reproducing the four rules would additionally require explicit per-reading evidence and target-linked witnesses. Section 9 outlines this replication as future work.

### 7.3 Replay Cost and Pipeline Resources

Replaying a deterministic rule requires no new model call, but deploying the hybrid gate adds an extraction stage to the two-call REVIEW pipeline. On Gemini 2.5 Flash, mean summed stage time rises from 3.00 to 4.92 seconds (64%); this is not a serving-latency measurement. The benefit to weigh against that cost depends on the reference (Finding 3). The added stage also demands more

of the generator: Llama 3.1 8B produced 68 invalid final outputs on the controlled path and fell outside the panel (Section 6.5), so the extraction stage should be validated for each generator before deployment. Larger memories would also require scoping witnesses to the records targeted by a proposal.

## 8 Limitations

This study uses specification-based replay to identify and characterize memory-write mechanisms within the evaluated pipelines, rather than to estimate their prevalence across deployed systems. Holding the extraction, proposal, initial state, executor, and reference fixed while varying admission rules allows differences in executed-state outcomes to be attributed to admission within this setting. The following considerations define the evidential and practical scope of these comparisons.

*Evidence base.* The main source comprises 48 authored Korean cases, complemented by 48 DSTC2 turns and three diagnostic probe sets—a contrast set, a boundary probe, and a MultiWOZ 2.4 probe (Appendices D–F). The alternative readings in the main source are written text that plays the role of competing recognition hypotheses; DSTC2 supplies hypotheses produced by a speech recognizer (Section 6.7). Twelve distinct main cases reach $H$. Recovered corrections occur in five of these cases, and additional protection from the evidence checks in the Gemini configurations occurs in three; outside DSTC2, witness-only holds comprise four observations from three situations. Configurations and repetitions reuse these cases, so distinct-case counts are reported alongside pooled counts to distinguish the breadth of the evidence from repeated observations of a mechanism. The revised extraction instruction was evaluated on five selected cases, with a zero-exposure configuration as a control (Appendix J.1). This provides a targeted test of the extraction-omission account on those cases. The replay analyses, the later Gemini configurations, and the 21-configuration panel criterion were defined after the initial results had been inspected; they explain those results, while the internally prespecified comparison keeps its original inferential status (Section 5.4, Table 5). The external dialogue sources broaden input coverage, while all replays retain our writer architecture. Transfer to other writers, estimates of mechanism prevalence, and broader comparisons of admission rules remain targets for replication (Section 9).

*Cost of control.* The complete-pipeline comparison and the fixed-proposal comparison measure different effects. Under the exact reference, positive-change completion is 1.30 percentage points lower for the controlled path than for Review (95% case-cluster interval −3.40 to +0.79). The corresponding interval under the variant-tolerant reference excludes zero (Section 6.5; Appendix K, Table K4). These estimates concern complete pipelines, in which generation as well as admission can differ. Separately, applying the full admission stage to fixed Review proposals yields 82 fewer reference violations and 40 fewer completed positive changes under the exact reference (Section 6.5, Table 8). These two counts quantify the writes the gate withholds, both unwanted and required, under the specified reference. Because exact-reference violations can include representational differences, the two counts are not a utility-weighted balance of harmful and useful writes. Their practical interpretation depends on the reference definition and on the relative costs of admitting an unwanted write and withholding a required one.

*References and validation.* Each held-out reference was checked separately by the two co-first authors. These internal reference checks are complemented by references derived from external annotations in DSTC2 and MultiWOZ 2.4. The authors designed the main cases and references, with AI assistance in drafting their text (see Data, Code, and AI Use). The contrast set was drafted by a separate AI agent following the authors' specification, without access to the rule code or earlier outputs. This separates its drafting from those experimental artifacts while retaining the author-defined task specification. Witness polarity remains a model judgment: structural and source-link checks verify the witness's form and linkage, rather than independently establishing semantic

support. The variant-tolerant evaluation is interpreted as a reference-sensitivity analysis. External annotation with reported agreement statistics, together with human assessment of the 38 candidate variant pairs, would extend validation of the references and the treatment of semantic equivalence (Section 9). The present evaluation concerns immediate memory-write decisions; semantic safety, user utility, downstream retrieval, and longitudinal interaction require separate evaluation.

## 9 Conclusion and Future Work

Specification-based replay separates what a single completion score merges: differences in generation, decisions by the admission rule, and conventions of the reference. It holds saved extractions and proposals fixed, applies each admission rule to the same artifacts, and scores the executed whole state against an explicit specification; once the required model outputs are cached, deterministic admission replay and rescoring require no further model calls. Exposure, counted from gate logs before scoring, bounds what a rule comparison can reveal: by Proposition 4.1, the 49 exposed controlled-path outputs limit any all-case difference among the four correction rules on the 3,024 panel episodes to 1.6 percentage points. Together, replay and exposure answer three questions that a completion score leaves open: which stage held a write, whether an admission rule was ever exercised, and how much a measured gain depends on the reference.

Applied to 21 model configurations, replay yielded three diagnostic findings. First, extraction omissions created recoverable holds: the 11 corrections recovered on Gemini 2.5 Flash were the same 11 that releasing every eligible held correction recovers, their holds originated in empty extracted fact lists, and a revised extraction instruction let the unchanged gate complete five of five selected corrections, against one of five under the original instruction. Second, zero exposure explained many rule ties: 12 of 21 configurations never produced an output on which the correction rules could decide differently, and only 49 controlled-path outputs from 12 cases reached that boundary. Third, reference conventions reshaped the gating trade-off: on saved Review proposals, accepting candidate tuple variants reduced the gate's avoided reference violations from 82 to 30 and increased its lost completed changes from 40 to 92. Together, these findings explain why selected component gains did not carry over to the internally prespecified complete-pipeline comparison, and the decomposition attributes the completion cost mainly to the gate's held appends rather than to the extraction-informed proposals.

The procedure diagnoses rather than presupposes a failure mechanism. Target-linked witnesses blocked values that appeared in every reading but belonged to another person or were negated, changing the executed state where literal occurrence could not. For a learned verifier, replay traced the additional completed corrections to the extraction-induced holds of the first finding. On real DSTC2 recognition hypotheses, the same trace returned a different cause, disagreement or incomplete support across hypotheses, showing that replay separates causes that an end-to-end score would merge.

These results change what an evaluation of write-admission policies should report. Among the eleven memory-write evaluations examined in Table 1, six score only downstream answers, and none reports the opportunity for the compared policies to differ. For attributing an improvement to a write-admission policy, each finding therefore yields a reporting check: trace every held write to the stage that produced it, report each rule's exposure alongside any comparison between rules, and rescore under alternative reference conventions. Together, these checks distinguish admission effects from generation differences, establish whether the comparison can discriminate between rules, and reveal sensitivity to the reference. They can be adapted to other memory writers when candidate operations, their supporting context, and the pre-write state can be recorded and replayed independently of generation; Section 7.2 outlines such an interface for Mem0, which separates extraction from update decisions.

*Future work.* The released cases, saved outputs, executed states, and evaluator (see Data, Code, and AI Use) make the next steps concrete. The first is independent human validation. For the four correction rules on the main panel, review can start from the 49 controlled-path and 23 review-proposal exposed outputs, once the missing ALLOW replays on 14 outputs are completed, together with the 38 candidate variant pairs; score ties alone do not justify excluding an output, and gating-versus-not-gating and learned-verifier comparisons need review sets of their own. A prospective protocol should fix the review population, contextual equivalence criteria, blinding to model and rule identities, disagreement handling, and analysis before judgments are collected, and report agreement before adjudication; any public registration would cover that new study. The second is replication on externally authored cases and on another memory writer frozen at its native interface (Section 7.2), with extraction revisions and admission thresholds fixed on development data before testing. The third is exposure-guided sampling: cases that distinguish the rules would sharpen rule comparisons, provided this enriched sampling is reported separately from prevalence estimation. Together, these steps would establish how far the three findings hold beyond our data.

## Data, Code, and AI Use

The archive at https://github.com/tzztqfgcsf-cloud/memory-write-replay contains the Korean test cases, references, prompts, raw outputs and failures, executed states, the evaluator, and the analysis code, including the variant-tolerant re-scoring, with provenance hashes; for DSTC2, which is used under its own terms and not redistributed, it contains the selected turn identifiers and the code that reconstructs the inputs. The experiments reported here involved no human participants. Generative AI tools (ChatGPT, Codex) were used under the authors' direction: to draft test-case text from the authors' specifications (Section 5.1), to edit language and organize the manuscript, and to assist with analysis code. The authors designed the study, the cases, and the analyses, reviewed all AI-assisted content, and take full responsibility for it. English examples are translations of the Korean inputs that define the experiments.

## References

[1] Vaibhav Adlakha, Parishad BehnamGhader, Xing Han Lu, Nicholas Meade, and Siva Reddy. 2024. Evaluating Correctness and Faithfulness of Instruction-Following Models for Question Answering. *Transactions of the Association for Computational Linguistics* 12 (2024), 681–699. doi:10.1162/tacl_a_00667

[2] Anirudh Ajith, Chris Pan, Mengzhou Xia, Ameet Deshpande, and Karthik Narasimhan. 2024. InstructEval: Systematic Evaluation of Instruction Selection Methods. In *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics, 4336–4350. doi:10.18653/v1/2024.findings-naacl.270

[3] Jacob Andreas, John Bufe, David Burkett, Charles Chen, Josh Clausman, Jean Crawford, Kate Crim, Jordan DeLoach, Leah Dorner, Jason Eisner, et al. 2020. Task-Oriented Dialogue as Dataflow Synthesis. *Transactions of the Association for Computational Linguistics* 8 (2020), 556–571. doi:10.1162/tacl_a_00333

[4] Sanghwan Bae, Donghyun Kwak, Soyoung Kang, Min Young Lee, Sungdong Kim, Yuin Jeong, Hyeri Kim, Sang-Woo Lee, Woomyoung Park, and Nako Sung. 2022. Keep Me Updated! Memory Management in Long-term Conversations. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 3769–3787. doi:10.18653/v1/2022.findings-emnlp.276

[5] Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. 2020. With Little Power Comes Great Responsibility. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 9263–9274. doi:10.18653/v1/2020.emnlp-main.745

[6] Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. 2026. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid? arXiv:2605.06527 doi:10.48550/arXiv.2605.06527

[7] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems*, Vol. 37. 130185–130213. doi:10.52202/079017-4136

[8] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413 doi:10.48550/arXiv.2504.19413

[9] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory Injection Attacks on LLM Agents via Query-Only Interaction. In *Advances in Neural Information Processing Systems*, Vol. 38. https://openreview.net/forum?id=i7J62t2wtV

[10] Yijiang River Dong, Tiancheng Hu, Zheng Hui, Caiqi Zhang, Ivan Vulić, Andreea Bobu, and Nigel Collier. 2026. Value of Information: A Framework for Human–Agent Communication. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 42879–42896. doi:10.18653/v1/2026.acl-long.1987

[11] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems*. ACM, 101–109. doi:10.1145/3298689.3347058

[12] Felix Gervits, Gordon Briggs, Antonio Roque, Genki A. Kadomatsu, Dean Thurston, Matthias Scheutz, and Matthew Marge. 2021. Decision-Theoretic Question Generation for Situated Reference Resolution: An Empirical Study and Computational Model. In *Proceedings of the 2021 International Conference on Multimodal Interaction*. ACM, 150–158. doi:10.1145/3462244.3479925

[13] Shengyue Guan, Jindong Wang, Jiang Bian, Bin Zhu, Jian-Guang Lou, and Haoyi Xiong. 2026. Evaluating LLM-based Agents for Multi-turn Conversations: A Survey. *ACM Transactions on Intelligent Systems and Technology* 17, 4, Article 85 (2026), 40 pages. doi:10.1145/3793671

[14] Jinyu Guo, Zhaokun Wang, Jingwen Pu, Wenhong Tian, Guiduo Duan, and Guangchun Luo. 2025. Multi-Perspective Dialogue Non-Quota Selection with loss monitoring for dialogue state tracking. *Expert Systems with Applications* 283 (2025), 127516. doi:10.1016/j.eswa.2025.127516

[15] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel Bowman, and Noah A. Smith. 2018. Annotation Artifacts in Natural Language Inference Data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. Association for Computational Linguistics, 107–112. doi:10.18653/v1/N18-2017

[16] Matthew Henderson, Blaise Thomson, and Jason D. Williams. 2014. The Second Dialog State Tracking Challenge. In *Proceedings of the 15th Annual Meeting of the Special Interest Group on Discourse and Dialogue (SIGDIAL)*. Association for Computational Linguistics, Philadelphia, PA, USA, 263–272. doi:10.3115/v1/W14-4337

[17] Paul W. Holland. 1986. Statistics and Causal Inference. *J. Amer. Statist. Assoc.* 81, 396 (1986), 945–960. doi:10.1080/01621459.1986.10478354

[18] Or Honovich, Roee Aharoni, Jonathan Herzig, Hagai Taitelbaum, Doron Kukliansy, Vered Cohen, Thomas Scialom, Idan Szpektor, Avinatan Hassidim, and Yossi Matias. 2022. TRUE: Re-evaluating Factual Consistency Evaluation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Seattle, United States, 3905–3920. doi:10.18653/v1/2022.naacl-main.287

[19] Sungdong Kim, Sohee Yang, Gyuwan Kim, and Sang-Woo Lee. 2020. Efficient Dialogue State Tracking by Selectively Overwriting Memory. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 567–582. doi:10.18653/v1/2020.acl-main.53

[20] Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua. 2025. Hello Again! LLM-powered Personalized Agent for Long-term Dialogue. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 5259–5276. doi:10.18653/v1/2025.naacl-long.272

[21] Hongjun Liu and Chen Zhao. 2026. Memory Is a Derivation: The Distributed-Evidence Paradox in Long-Term Agents. arXiv:2609.36130 doi:10.48550/arXiv.2609.36130

[22] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[23] Zhanghui Liu, Chaoxiong Zhou, Yuzhong Chen, and Zeping Zheng. 2026. Few-shot dialogue state tracking via context ranking and dynamic contrastive decoding. *Expert Systems with Applications* 300 (2026), 130197. doi:10.1016/j.eswa.2025.130197

[24] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46534–46594. doi:10.52202/075280-2019

[25] Chris Madge, Matthew Purver, and Massimo Poesio. 2025. Referential ambiguity and clarification requests: comparing human and LLM behaviour. In *Proceedings of the Eighth Workshop on Computational Models of Reference, Anaphora and Coreference*. Association for Computational Linguistics, 1–11. doi:10.18653/v1/2025.crac-1.1

[26] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 13851–13870. doi:10.18653/v1/2024.acl-long.747

[27] Felipe Maia Polo, Subha Maity, Mikhail Yurochkin, Moulinath Banerjee, and Yuekai Sun. 2024. Weak Supervision Performance Evaluation via Partial Identification. In *Advances in Neural Information Processing Systems*, Vol. 37.

[28] Kevin Musgrave, Serge Belongie, and Ser-Nam Lim. 2020. A Metric Learning Reality Check. In *Computer Vision – ECCV 2020 (Lecture Notes in Computer Science, Vol. 12370)*. Springer, 681–699. doi:10.1007/978-3-030-58595-2_41

[29] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 doi:10.48550/arXiv.2310.08560

[30] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. 2025. SeCom: On Memory Construction and Retrieval for Personalized Conversational Agents. In *International Conference on Learning Representations*, Vol. 2025. 91851–91885. https://proceedings.iclr.cc/paper_files/paper/2025/hash/e56f394bbd4f0ec81393d767caa5a31b-Abstract-Conference.html

[31] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 1–22. doi:10.1145/3586183.3606763

[32] Sudha Rao and Hal Daumé III. 2018. Learning to Ask Good Questions: Ranking Clarification Questions using Neural Expected Value of Perfect Information. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2737–2746. doi:10.18653/v1/P18-1255

[33] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. 2023. Measuring Attribution in Natural Language Generation Models. *Computational Linguistics* 49, 4 (2023), 777–840. doi:10.1162/coli_a_00486

[34] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956 doi:10.48550/arXiv.2501.13956

[35] Wout Schellaert, Fernando Martínez-Plumed, and José Hernández-Orallo. 2025. Analysing the Predictability of Language Model Performance. *ACM Transactions on Intelligent Systems and Technology* 16, 2, Article 30 (2025), 26 pages. doi:10.1145/3706118

[36] Ivan Sekulić, Mohammad Aliannejadi, and Fabio Crestani. 2024. Analysing Utterances in LLM-Based User Simulation for Conversational Search. *ACM Transactions on Intelligent Systems and Technology* 15, 3, Article 62 (2024), 22 pages. doi:10.1145/3650041

[37] Jaineet Shah. 2026. Causal Agent Replay: Counterfactual Attribution for LLM-Agent Failures. arXiv:2606.08275 doi:10.48550/arXiv.2606.08275

[38] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. doi:10.52202/075280-0377

[39] Leila Tavakoli, Johanne R. Trippas, Hamed Zamani, Falk Scholer, and Mark Sanderson. 2025. Online and Offline Evaluation in Search Clarification. *ACM Transactions on Information Systems* 43, 1, Article 2 (2025), 30 pages. doi:10.1145/3681786

[40] Dechuan Teng, Zheng Yan, and Wanxiang Che. 2026. Program of actions (PoA): Interactive semantic action parser as more powerful dialogue state tracker through code generation. *Expert Systems with Applications* 301 (2026), 130481. doi:10.1016/j.eswa.2025.130481

[41] Alberto Testoni and Raquel Fernández. 2024. Asking the Right Question at the Right Time: Human and Model Uncertainty Guidance to Ask Clarification Questions. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 258–275. doi:10.18653/v1/2024.eacl-long.16

[42] Xin Tian, Liankai Huang, Yingzhan Lin, Siqi Bao, Huang He, Yunyi Yang, Hua Wu, Fan Wang, and Shuqi Sun. 2021. Amendable Generation for Dialogue State Tracking. In *Proceedings of the 3rd Workshop on Natural Language Processing for Conversational AI*. Association for Computational Linguistics, 80–92. doi:10.18653/v1/2021.nlp4convai-1.8

[43] Bo-Hsiang Tseng, Shruti Bhargava, Jiarui Lu, Joel Ruben Antony Moniz, Dhivya Piraviperumal, Lin Li, and Hong Yu. 2021. CREAD: Combined Resolution of Ellipses and Anaphora in Dialogues. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 3390–3406. doi:10.18653/v1/2021.naacl-main.265

[44] Qingyue Wang, Yanhe Fu, Yanan Cao, Shuai Wang, Zhiliang Tian, and Liang Ding. 2025. Recursively summarizing enables long-term dialogue memory in large language models. *Neurocomputing* 639 (2025), 130193. doi:10.1016/j.neucom.2025.130193

[45] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=1PL1NIMMrw

[46] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations*, Vol. 2025. 86809–86836. https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[47] Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. 2024. Knowledge Conflicts for LLMs: A Survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 8541–8565. doi:10.18653/v1/2024.emnlp-main.486

[48] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., Red Hook, NY, USA, 17577–17604. doi:10.52202/085713-0593

[49] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. 2026. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12805–12825. doi:10.18653/v1/2026.acl-long.583

[50] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. $\tau$-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations (ICLR)*. 9965–10017. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html

[51] Fanghua Ye, Jarana Manotumruksa, and Emine Yilmaz. 2022. MultiWOZ 2.4: A Multi-Domain Task-Oriented Dialogue Dataset with Essential Annotation Corrections to Improve State Tracking Evaluation. In *Proceedings of the 23rd Annual Meeting of the Special Interest Group on Discourse and Dialogue*. Association for Computational Linguistics, Edinburgh, UK, 351–360. doi:10.18653/v1/2022.sigdial-1.34

[52] Steve Young, Milica Gašić, Blaise Thomson, and Jason D. Williams. 2013. POMDP-Based Statistical Spoken Dialog Systems: A Review. *Proc. IEEE* 101, 5 (2013), 1160–1179. doi:10.1109/JPROC.2012.2225812

[53] Xiaoxue Zang, Abhinav Rastogi, Srinivas Sunkara, Raghav Gupta, Jianguo Zhang, and Jindong Chen. 2020. MultiWOZ 2.2: A Dialogue Dataset with Additional Annotation Corrections and State Tracking Baselines. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*. Association for Computational Linguistics, 109–117. doi:10.18653/v1/2020.nlp4convai-1.13

[54] Guilin Zhang, Wei Jiang, Xiejiashan Wang, Aisha Behr, Kai Zhao, Jeffrey Friedman, Xu Chu, and Amine Anoun. 2026. Adaptive Memory Admission Control for LLM Agents. arXiv:2603.04549 doi:10.48550/arXiv.2603.04549

[55] Michael JQ Zhang and Eunsol Choi. 2025. Clarify When Necessary: Resolving Ambiguity Through Interaction with LMs. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 5541–5558. doi:10.18653/v1/2025.findings-naacl.306

[56] Yan Zhang and Shibo Li. 2026. ConsistencyGate: Preventing Memory Contamination in LLM Agents via Self-Consistency Admission Control. arXiv:2607.22962 doi:10.48550/arXiv.2607.22962

[57] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46595–46623. doi:10.52202/075280-2020

[58] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 17 (2024), 19724–19731. doi:10.1609/aaai.v38i17.29946